Machine Learning Lab - Programs 1 to 10

Program 1

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .csv file.

Code:

```
import csv

with open("data.csv", "r") as file:
    data = list(csv.reader(file))

header = data[0]
examples = data[1:]
hypothesis = ["?"] * (len(header) - 1)

for row in examples:
    if row[-1] == "yes":
        for i in range(len(hypothesis)):
            if hypothesis[i] == "0":
                hypothesis[i] = row[i]
            elif hypothesis[i] != row[i]:
                hypothesis[i] = "?"

print("Most Specific Hypothesis:")
print(hypothesis)
```

Output:

```
Most Specific Hypothesis:
['?', '?', '?', '?']
```

Program 2

For a given set of training data examples stored in a .csv file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with training examples.

Code:

```
import pandas as pd

data = pd.read_csv("data.csv")
S = ['0', '0', '0', '0']
G = ['?', '?', '?', '?']

for i in range(len(data)):
    x = list(data.iloc[i, :-1])
    y = data.iloc[i, -1]
    if y == "yes":
        for j in range(4):
            if S[j] == '0':
                S[j] = x[j]
            elif S[j] != x[j]:
                S[j] = '?'

print("Specific Boundary:", S)
print("General Boundary:", G)
```

Output:

```
Specific Boundary : ['Sunny', 'Warm', '?', '?']
General Boundary : ['?', '?', '?', '?']
```

Program 3

Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Code:

```
import csv
from sklearn.tree import DecisionTreeClassifier

with open("play.csv", "r") as file:
    data = list(csv.reader(file))

x = []
y = []

for row in data[1:]:
    x.append([0 if row[0] == "Sunny" else 1])
    y.append(0 if row[1] == "NO" else 1)

model = DecisionTreeClassifier(criterion="entropy")
model.fit(x, y)
prediction = model.predict([[0]])

if prediction[0] == 1:
    print("Play Tennis : Yes")
else:
    print("Play Tennis : NO")
```

Output:

```
Play Tennis : NO
```

Program 4

Write a program to implement the naive Bayesian classifier for a sample training data set stored as a csv file. Compute the accuracy of the classifier, considering few test data sets.

Code:

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score

data = pd.read_csv("data.csv")
le = LabelEncoder()
x = le.fit_transform(data["Weather"]).reshape(-1, 1)
y = le.fit_transform(data["play"])

model = GaussianNB()
model.fit(x, y)
pred = model.predict(x)

print("Accuracy:", accuracy_score(y, pred) * 100, "%")
```

Output:

```
Accuracy : 83.33333333333334 %
```

Program 5

Assuming a set of documents that need to be classified, use the naive Bayesian Classifier model to perform this task. Calculate the accuracy, precision and recall for your data set.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

data = pd.read_csv("data.csv")
x = data["text"]
y = data["label"]

vectorizer = CountVectorizer()
x = vectorizer.fit_transform(x)
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=42
)

model = MultinomialNB()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, pos_label="positive"))
print("Recall :", recall_score(y_test, y_pred, pos_label="positive"))
```

Output:

```
Accuracy : 0.25
Precision : 0.0
Recall : 0.0
```

Program 6

Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using Standard Heart Disease Data set.

Code:

```
import pandas as pd
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.inference import VariableElimination

data = pd.read_csv("heart.csv")
data = data[["age", "cp", "thalach", "target"]]

data["age"] = pd.cut(
    data["age"],
    bins=[0, 40, 60, 100],
    labels=["Young", "Middle", "Old"]
)

data["thalach"] = pd.cut(
    data["thalach"],
    bins=[0, 120, 160, 250],
    labels=["low", "Normal", "High"]
)

model = DiscreteBayesianNetwork([
    ("age", "target"),
    ("cp", "target"),
    ("thalach", "target")
])
model.fit(data)

inference = VariableElimination(model)
result = inference.query(
    variables=["target"],
    evidence={
        "age": "Middle",
        "cp": 2,
        "thalach": "Normal"
    }
)

print(result)
print("\nDiagnosis:")
if result.values[1] > result.values[0]:
    print("Heart Disease")
else:
    print("NO Heart Disease")
```

Output:

```
+-----------+---------------+
| target    |   phi(target) |
+===========+===============+
| target(0) |        0.2963 |
+-----------+---------------+
| target(1) |        0.7037 |
+-----------+---------------+

Diagnosis:
Heart disease
```

Program 7

Apply EM algorithm to cluster a set of data stored in a .csv file.

Code:

```
import pandas as pd
from sklearn.mixture import GaussianMixture

data = pd.read_csv("data.csv")
model = GaussianMixture(n_components=2)
model.fit(data)
data["Cluster"] = model.predict(data)
print(data)
```

Output:

```
   Height  Weight  Cluster
0     150      50        0
1     152      52        0
2     155      54        0
3     158      56        0
4     160      58        0
5     180      80        1
6     182      82        1
7     185      85        1
8     188      87        1
9     190      90        1
```

Program 8

Use the .csv file data set for clustering using k-means algorithm.

Code:

```
import pandas as pd
from sklearn.cluster import KMeans

data = pd.read_csv("data.csv")
x = data[["Age", "Salary"]]
kmeans = KMeans(n_clusters=2, random_state=0, n_init=10)
data["cluster"] = kmeans.fit_predict(x)
print(data)
```

Output:

```
   Age  Salary  Cluster
0   20   20000        1
1   22   22000        1
2   21   21000        1
3   25   25000        1
4   24   24000        1
5   45   60000        0
6   48   65000        0
7   50   70000        0
8   46   62000        0
9   52   72000        0
```

Program 9

Write a program to implement K-Nearest Neighbour algorithm to classify the IRIS data set. Print both correct and wrong predictions.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
x = iris.data
y = iris.target
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)

for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]

    if y_test[i] == y_pred[i]:
        print("Correct:", "Actual =", actual, ", Predicted =", predicted)
    else:
        print("Wrong:", "Actual =", actual, ", Predicted =", predicted)
```

Output:

```
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = virginica , Predicted = virginica
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = versicolor , Predicted = versicolor
Correct: Actual = setosa , Predicted = setosa
Correct: Actual = setosa , Predicted = setosa
```

Program 10

Implement the non-parametric locally weighted Regression algorithm in order to fit data points. Select appropriate data set for experiment and draw graphs.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
y = np.array([25, 30, 42, 48, 50, 65, 70, 75, 90, 105])

def predict(point):
    weights = np.exp(-((x - point) ** 2) / 8)
    mean_x = np.sum(weights * x) / np.sum(weights)
    mean_y = np.sum(weights * y) / np.sum(weights)
    numerator = np.sum(weights * (x - mean_x) * (y - mean_y))
    denominator = np.sum(weights * (x - mean_x) ** 2)
    slope = numerator / denominator
    intercept = mean_y - slope * mean_x
    return slope * point + intercept

predicted = []
for point in x:
    predicted.append(predict(point))

plt.scatter(x, y, color="blue", label="Actual Data")
plt.plot(x, predicted, color="red", label="LWR fitted curve")
plt.xlabel("years of experience")
plt.ylabel("salary")
plt.title("locally weighted Regression")
plt.legend()
plt.grid()
plt.show()
```

Output:

